Kasi Yedumati · Full Stack Training

Joins

Module Outcome:

- Join students to courses (or orders to customers) instead of one giant table
- Tell INNER from LEFT — missing matches stay or vanish on purpose
- Read a join result and know which table each column came from

So far we have learnt to analyse the data that is present in a single table. But in real-world scenarios, often the data is distributed in multiple tables. To fetch meaningful insights, we have to bring the data together by 'combining' the tables.

We use **JOIN** clause to combine rows from two or more tables, based on a related column between them. Types of joins: Natural join, Inner join, Full Join, Cross Join, Left Join, Right Join.

Database: The database stores the data of students, courses, course reviews, instructors, etc. of an e-learning platform.

```
instructor --1-- teaches --n-- course
course --1-- has --n-- reviews
student --n-- Enroll --n-- course
student --1-- create --n-- reviews
```

Natural Join

NATURAL JOIN combines the tables based on the common columns.

Syntax:

```
SELECT *
FROM table1
NATURAL JOIN table2;
```

Example: Fetch the details of courses that are being taught by 'Alex'.

```
SELECT
    course_name,
    instructor_full_name
FROM
    course
NATURAL JOIN instructor
WHERE
    instructor_full_name = "Alex";
```

Output:

name	full_name
Cyber Security	Alex

Inner Join

INNER JOIN combines rows from both the tables if they meet a specified condition.

Syntax:

```
SELECT *
FROM table1
INNER JOIN table2
ON table1.c1 = table2.c2;
```

Note: We can use any comparison operator in the condition.

Example: Get the reviews of course "Cyber Security" (course with id = 15).

```
SELECT
    student.full_name,
    review.content,
    review.created_at
FROM
    student
INNER JOIN review
ON
    student.id = review.student_id
WHERE
    review.course_id = 15;
```

Output:

full_name	content	created_at
Ajay	Good explanation	2021-01-19
Ajay	cyber security is awesome	2021-01-20

Left Join

In LEFT JOIN, for each row in the left table, matched rows from the right table are combined. If there is no match, NULL values are assigned to the right half of the rows in the temporary table.

Syntax:

```
SELECT *
FROM table1
LEFT JOIN table2
ON table1.c1 = table2.c2;
```

Example: Fetch the full name of students who have not enrolled for any course.

```
SELECT
    student.full_name
FROM
    student
LEFT JOIN student_course
```

```
ON
    student.id = student_course.student_id
WHERE
    student_course.id IS NULL;
```

Output:

full_name
Afrin

Right Join

RIGHT JOIN or RIGHT OUTER JOIN is vice versa of LEFT JOIN i.e., in RIGHT JOIN, for each row in the right table, matched rows from the left table are combined. If there is no match, NULL values are assigned to the left half of the rows in the temporary table.

Syntax:

```
SELECT *
FROM table1
RIGHT JOIN table2
ON table1.c1 = table2.c2;
```

Example: Perform RIGHT JOIN on course and instructor tables.

```
SELECT
    course.name,
    instructor.full_name
FROM
    course
RIGHT JOIN instructor
ON
    course.instructor_id = instructor.instructor_id;
```

Note: RIGHT JOIN is supported in MySQL. Some DBMS such as SQLite do not support RIGHT JOIN.

Full Join

FULL JOIN or FULL OUTER JOIN is the result of both RIGHT JOIN and LEFT JOIN.

Syntax:

```
SELECT *
FROM table1
FULL JOIN table2
ON table1.c1 = table2.c2;
```

Example:

```
SELECT
    course.name,
```

```
    instructor.full_name
FROM
    course
FULL JOIN instructor
ON
    course.instructor_id = instructor.instructor_id;
```

Note: FULL JOIN is not supported in MySQL. Use the **UNION workaround** below.

MySQL workaround for FULL OUTER JOIN:

```
SELECT course.name, instructor.full_name
FROM course
LEFT JOIN instructor
ON course.instructor_id = instructor.id
UNION
SELECT course.name, instructor.full_name
FROM course
RIGHT JOIN instructor
ON course.instructor_id = instructor.id;
```

`UNION` combines the LEFT JOIN result (all courses) with unmatched instructors from the RIGHT JOIN, giving the same effect as FULL OUTER JOIN.

Cross Join

In CROSS JOIN, each row from the first table is combined with all rows in the second table. Cross join is also called **CARTESIAN JOIN**.

Syntax:

```
SELECT *
FROM table1
CROSS JOIN table2;
```

Example:

```
SELECT
    course_name AS course_name,
    instructor_full_name AS instructor_name
FROM
    course
CROSS JOIN instructor;
```

Output:

course_name	instructor_name
Machine Learning	Alex
Machine Learning	Ann
Cyber Security	Alex

Self Join

We can combine a table with itself. This kind of join is called **SELF JOIN**.

Syntax:

```
SELECT t1.c1,
       t2.c2
FROM table1 AS t1
JOIN table1 AS t2
ON t1.c1 = t2.c1;
```

Note: We can use any JOIN clause in self-join.

Example: Get student pairs who registered for common course.

```
SELECT
    sc1.student_id AS student_id1,
    sc2.student_id AS student_id2,
    sc1.course_id
FROM
    student_course AS sc1
    INNER JOIN student_course sc2
    ON sc1.course_id = sc2.course_id
WHERE
    sc1.student_id < sc2.student_id;
```

Output:

student_id1	student_id2	course_id
1	3	11

Joins on Multiple Tables

We can also perform join on a combined table.

Example: Fetch all the students who enrolled for the courses taught by the instructor "Arun" (id = 102).

```
SELECT
    T.name AS course_name,
    student.full_name
FROM
    (course
    INNER JOIN student_course
    ON course.id = student_course.course_id) AS T
INNER JOIN student
ON T.student_id = student.id
WHERE
    course.instructor_id = 102;
```

Output:

course_name	full_name
machine learning	Varun
machine learning	Sandya

Best practices:

1. Use ALIAS to name the combined table.
2. Use alias table names to refer the columns in the combined table.

Using Joins with Other Clauses

We can apply WHERE, ORDER BY, HAVING, GROUP BY, LIMIT and other clauses on joined tables as well.

Example: Get the name of the student who scored highest in 'Machine Learning' course.

```
SELECT
    student.full_name
FROM
    (course
    INNER JOIN student_course
    ON course.id = student_course.course_id) AS T
    INNER JOIN student
    ON T.student_id = student.id
WHERE
    course.name = "Machine Learning"
ORDER BY
    student_course.score DESC
LIMIT 1;
```

Using Joins with Aggregations

We can apply aggregate functions such as SUM, AVG, COUNT, MAX, MIN on the temporary joined-table as well.

Example: Get the highest score in each course.

```
SELECT
    course.name AS course_name,
    MAX(score) AS highest_score
FROM
    course LEFT JOIN student_course
    ON course.id = student_course.course_id
GROUP BY
    course.id;
```

Joins Summary

Join Type	Use Case
Natural join	Joins based on common columns

Join Type	Use Case
Inner join	Joins based on a given condition
Left join	All rows from left table and matched rows from right table
Right join	All rows from right table and matched rows from left table
Full join	All rows from both the tables
Cross join	All possible combinations